

CUDA-Accelerated Simulation of Variational Quantum Circuits: A High-Performance Framework for Data-Conditioned VQE Benchmarking

Juan Pablo Acuña González, Moisés Sánchez Adame, Oscar Montiel*

Abstract The computational cost of simulating variational quantum algorithms remains a major bottleneck for their deployment in practical settings. This work presents a CUDA-accelerated framework for efficient simulation of variational quantum circuits, with a focus on data-conditioned variational quantum eigensolvers, where quantum states are fixed via classical data encoding and optimization is performed over Hamiltonian parameters. We develop a GPU-based implementation for gate-level operations by executing the full variational circuit in a single GPU call, minimizing memory transfers and enabling efficient statevector updates and parallelized expectation value computation. On a 3-qubit system, the framework is evaluated as a controlled testbed to analyze computational performance, memory efficiency, and numerical stability. Benchmarking experiments compare CPU and GPU implementations across multiple configurations, including varying numbers of Hamiltonian terms, measurement shots, and dataset sizes. Results show consistent reductions in execution time and improved efficiency in iterative optimization loops, highlighting the benefits of GPU acceleration for hybrid quantum-classical workflows. This work characterizes the computational behavior of small-scale systems and discusses pathways toward scaling through architectural optimizations. A data-driven formulation is included as a case study to demonstrate integration with machine learning pipelines, providing a reproducible and extensible high-performance framework for quantum circuit simulation.

Juan Pablo Acuña González

Instituto Politécnico Nacional, CITEDI-IPN, Av, Instituto Politécnico Nacional, No. 1310, Colonia Nueva Tijuana, 22435 Tijuana, B.C., Mexico. e-mail: jagonzalez@citedi.mx

Moisés Sánchez Adame

Instituto Politécnico Nacional, CITEDI-IPN, Av, Instituto Politécnico Nacional, No. 1310, Colonia Nueva Tijuana, 22435 Tijuana, B.C., Mexico. e-mail: msanchez@citedi.mx

Oscar Montiel*

Instituto Politécnico Nacional, CITEDI-IPN, Av, Instituto Politécnico Nacional, No. 1310, Colonia Nueva Tijuana, 22435 Tijuana, B.C., Mexico. Corresponding author e-mail: oross@citedi.mx

1 Introduction

The variational quantum eigensolver (VQE) stands as a cornerstone of near-term quantum computing applications. Looking back across nearly a decade of hybrid quantum q-algorithm pipelines targeting complex physical systems, the field has evolved from a foundational conceptualization of variational feedback loops down to rigorous classical simulation engines capable of emulating noisy hardware constraints [1]. Early frameworks established that while a universal, fault-tolerant quantum machine requires error-free execution over millions of physical qubits, near-term noisy intermediate-scale quantum (NISQ) devices are heavily bound by limited coherence times, sparse hardware connectivity, and restricted gate sets [1]. This structural limitation led to the formalization of the hybrid quantum-classical paradigm, where a quantum co-processor handles highly entangled trial wavefunctions while a classical optimizer iteratively shifts gate parameters to minimize a physical cost function or an energy expectation value [1]. To evaluate these early physical processors against algorithmic demands, metrics such as the architecture-neutral quantum volume were established, mathematically linking physical qubit counts to allowable circuit depths before accumulated gate error or crosstalk masks the underlying statevector probabilities [1].

As these foundational models matured, the development of hybrid quantum computing pipelines progressed from idealized proof-of-concept studies toward tangible applications targeting real-world industrial and clinical challenges [2]. In the domain of pharmaceuticals, advanced hybrid workflows have successfully transitioned from abstract computational experiments to highly specialized engineering frameworks designed to capture the true intricacies of drug design, such as modeling covalent bond interactions and determining the precise Gibbs free energy profiles governing prodrug activation involving covalent bond cleavage [2]. In parallel, macro-level surveys within oncology and structural biomedicine underscore how high-performance computing (HPC) and quantum machine learning (QML) are converging to model complex disease landscapes [3]. The integration of quantum processing units (QPUs) as domain-specific accelerators within broader distributed architectures—a paradigm termed "quantum-centric supercomputing"—holds the theoretical potential to bypass classical algorithmic bottlenecks, leveraging polynomial or polylogarithmic complexity scaling to drastically compress time-to-solution metrics in multi-scale continuum simulations [3].

However, because physical NISQ hardware remains constrained by environmental noise and operational error rates, high-performance classical simulation has become an indispensable framework to validate these mature pipelines and design next-generation variational algorithms [4]. Classical simulation paradigms generally divide into full-amplitude statevector emulation and approximate Tensor Network (TN) contractions [5]. For massive chemical or complex biological systems, simulating full-amplitude statevectors requires extensive distributed HPC schemes to handle the exponential growth of the state space data across nodes ($N = 2^n$ complex coefficients for n qubits), imposing a strict $O(2^n)$ memory allocation floor that

induces severe Von Neumann memory-bandwidth bottlenecks during multi-qubit transformations [4, 5].

To mitigate these memory bounds, general-purpose graphics processing unit (GPGPU) aided simulation has emerged as the premier strategy for accelerating full-amplitude statevector evolution [5]. Early GPGPU emulation designs established that single-qubit unitary gate operations—which map onto highly decoupled, parallelizable data structures over independent pairs of amplitudes $(\alpha_i, \alpha_{i \oplus 2^k})$ —can achieve massive thread-level parallel efficiency, such as those governing the Quantum Fourier Transform (QFT) [6]. Recent macro-architectural advancements, such as the comprehensive parallelization implemented for the Q^2 Chemistry software package, utilize sophisticated strategies like batch-buffered overlap processing, dependency-aware gate contraction, and staggered multi-gate parallelism to optimize simulators across large-scale distributed CPU and GPU multi-node configurations [4].

Despite these classical scaling victories, a persistent architectural divide remains when validating algorithms intended for small-scale physical hardware platforms [5]. Small-scale physical systems, whether built on superconducting loops, trapped-ion strings, or neutral-atom arrays, are fundamentally bounded by hardware-specific constraints: superconducting qubits suffer from localized cross-talk and restricted nearest-neighbor 2D mesh layouts, while ion traps provide all-to-all connectivity at the cost of significantly slower gate execution speeds and susceptible vibrational-mode decoherence [1, 5]. In [5] the dominant acceleration taxonomy is formalized, charting the future directions of hardware-aware optimizations, demonstrating that to simulate physical circuits beyond the 30-qubit threshold on classical infrastructure, frameworks must actively explore lossy approximation methods, hardware-aware tensor edge-cutting, lossy amplitude truncation, and highly customized distributed scheduling paradigms to handle multi-node latency and high queuing overheads.

A profound algorithmic and architectural contrast arises when comparing the future simulation directions proposed by [5] against the data-conditioned variational framework developed in this book chapter. While standard simulation testbeds seek to achieve scaling longevity by sparsifying deep, complex quantum states to match macro-scale distributed systems, the approach presented in this work completely reverses this operational vector. Curated programmatically via the qvirus package ecosystem, our routine bypasses the need for massive tensor approximations or multi-node process coordination by implementing a hardware-optimized, data-conditioned inverse VQE pipeline focused on small-scale, high-throughput execution layouts.

Rather than executing a dynamic circuit ansatz to track the unknown, highly entangled ground state of a physical molecular Hamiltonian, which introduces an expensive parameter search space scaling as $O(N^4)$ for traditional unitary coupled cluster (UCC) expansions [1], our inverse formulation locks down the quantum state vector up front [7]. We programmatically map and encode normalized patient immunometabolic metrics directly into the rotational parameters of an extended quantum state preparation circuit [7]. This operational shift effectively transfers the downstream iterative optimization landscape away from the state space and entirely

onto discovering the parameters of a task-specific variational problem Hamiltonian whose coefficients are derived from multi-pathogen transcriptomic datasets.

Moving beyond the foundational 2-qubit clinical models that established initial mappings between distinct cell lineages and low-dimensional quantum state spaces [7], this work scales the formulation to an expressive multi-qubit configuration. By systematically evaluating how each patient-encoded quantum state fits into the energy landscape of a variety of candidate Hamiltonians, we utilize a delta metric representing the discrepancy between the estimated expectation value and the true minimum eigenvalue to classify complex clinical outcomes.

Where traditional simulation paradigms require multiple independent inter-kernel launches that induce severe device-host memory-transfer latency over physical PCIe buses [5], our framework compresses the entire variational circuit execution, state manipulation, and finite-shot expectation value aggregation into a single, fused GPU kernel call. By confining the statevector data entirely within the high-speed local register space of a single SM and substituting multi-stride memory access with bitwise index masking, the architecture completely eliminates the exponential data-transfer penalty for local qubit clusters. Our findings demonstrate that this data-conditioned inverse VQE routine serves as a powerful method for non-linear dimension reduction and high-throughput diagnostic classification, grounding the theoretical aspirations of quantum-centric biomedical modeling within a concrete, high-performance execution architecture.

2 CUDA-Accelerated Inverse VQE Testbed

To investigate the computational performance of GPU-accelerated variational quantum simulations, we constructed a controlled 3-qubit testbed implementing a data-conditioned inverse VQE. The objective of this framework is not to solve a specific physical Hamiltonian, but rather to evaluate the efficiency of a hybrid quantum-classical workflow in which quantum states are fixed by classical feature encoding and optimization is performed over Hamiltonian parameters.

The 3-qubit configuration was selected as the smallest architecture capable of simultaneously representing higher-order interactions, entanglement patterns, and non-trivial multi-body Pauli operators while maintaining complete statevector accessibility for detailed profiling and numerical validation. The resulting Hilbert space, $\mathcal{H} = (\mathbb{C}^2)^{\otimes 3} \cong \mathbb{C}^8$, contains eight computational basis states and provides a sufficiently expressive environment for benchmarking gate execution, expectation-value estimation, memory consumption, and optimization throughput.

Unlike conventional VQE formulations, where a parameterized quantum circuit is iteratively optimized to minimize the energy of a fixed Hamiltonian, the inverse formulation considered here assumes that the quantum state has already been specified through classical feature encoding. The optimization task is therefore transferred to the Hamiltonian construction stage, where candidate operators are evaluated ac-

cording to the discrepancy between their minimum eigenvalue and the expectation value induced by the encoded state.

This formulation naturally decouples quantum state preparation from Hamiltonian identification, making it particularly suitable for data-driven learning scenarios while simultaneously exposing a computational workload dominated by repeated state preparation and expectation-value evaluation. Consequently, the framework provides an ideal benchmark for studying the impact of GPU acceleration on variational quantum simulation pipelines.

2.1 Data Encoding

Each sample is represented by a set of 6 real-valued input features, corresponding to the 6 rotational degrees of freedom of the 3-qubit architecture. Prior to quantum state preparation, all variables are normalized to the interval $[0, 2\pi]$ through global min-max scaling. This transformation ensures a consistent mapping between the classical feature domain and the periodic parameter space of quantum rotation operators while preventing saturation effects caused by heterogeneous feature ranges. For a feature value x , the normalized angle is computed as:

$$x^* = \frac{x - \min_k(x_k)}{\max_k(x_k) - \min_k(x_k)} \cdot 2\pi, \quad \forall x \in \mathcal{F}, \quad (1)$$

where $\mathcal{F}$ denotes the selected feature set and the extrema are calculated over the complete dataset. Degenerate feature ranges are handled explicitly by assigning a zero rotation angle, thereby avoiding numerical instabilities associated with division-by-zero operations.

Following normalization, the feature space is mapped directly onto the rotational coordinates of the quantum circuit. For the 3-qubit architecture, 6 normalized variables are assigned to the polar and azimuthal angles of the individual qubits according to $\Theta = (\theta_0, \phi_0, \theta_1, \phi_1, \theta_2, \phi_2)$, where each qubit receives an independent pair of angular parameters:

$$\begin{aligned} Q_0 &\leftarrow (\theta_0, \phi_0), \\ Q_1 &\leftarrow (\theta_1, \phi_1), \\ Q_2 &\leftarrow (\theta_2, \phi_2). \end{aligned} \quad (2)$$

This assignment preserves local feature pairings prior to entanglement and provides a direct correspondence between classical variables and quantum degrees of freedom. The resulting parameter vector fully specifies the state-preparation layer and serves as input to the CUDA-accelerated ansatz kernel.

To eliminate redundant preprocessing during iterative optimization, all rotation angles are computed once on the host prior to circuit execution. Algorithm 1 summarizes the preprocessing procedure.

Algorithm 1 Host-Side Precomputation of Rotation Angles**Require:** Dataset D , selected feature subset $\mathcal{F}$

- 1: Compute global feature ranges $(\min(f), \max(f))$, $\forall f \in \mathcal{F}$
- 2: **for** each sample j in D **do**
- 3: **for** each feature $f_i \in \mathcal{F}$ **do**
- 4: Normalize using Eq. (1)
- 5: **end for**
- 6: Construct $(\theta_0, \phi_0, \theta_1, \phi_1, \theta_2, \phi_2)$
- 7: **end for**
- 8: Store resulting angle matrix in host memory **return** Precomputed angle matrix

The generated angle matrix is subsequently transferred to the quantum simulation stage, where each row is consumed by a single invocation of the CUDA statevector kernel. This separation between classical preprocessing and quantum execution minimizes host-device synchronization overhead and allows benchmarking results to reflect the performance of the simulation architecture itself rather than data preparation costs.

2.2 The Fused Ansatz Circuit

The encoded parameter vector obtained in Subsection 2.1 is transformed into a quantum state through a shallow variational circuit composed of a data-preparation layer followed by a fixed entangling topology. The preparation layer applies successive R_x and R_y rotations to each qubit, mapping the 6 normalized classical features onto the rotational degrees of freedom of the quantum register. To introduce higher-order correlations between encoded variables, the circuit employs a cyclic entanglement pattern forming a closed interaction loop across all 3 qubits. The complete state-preparation operator is therefore:

$$|\psi_j\rangle = CX_{2,0} CX_{1,2} CX_{0,1} \left(\bigotimes_{i=0}^2 R_y(\phi_{j,i}) R_x(\theta_{j,i}) \right) |0\rangle^{\otimes 3}. \quad (3)$$

Figure 1 illustrates the resulting 3-qubit circuit architecture. A CUDA routine executes the circuit, encapsulating the entire state-preparation workflow within a single host-side invocation. Starting from the initial basis state $|000\rangle$, the routine allocates the statevector in device memory, applies all parameterized rotations and controlled-NOT operations directly on the GPU, and returns the final statevector only after circuit completion. Although individual gate transformations are executed through dedicated GPU kernels, intermediate statevectors remain resident in device memory throughout the entire evolution. This design eliminates repeated host-device synchronization and avoids memory-transfer overhead between successive gate applications, allowing the benchmark to isolate the computational cost of quantum state evolution itself. Single-qubit rotations are generated dynamically from the encoded

parameters and applied through parallel amplitude updates based on bitwise index masking. The cyclic entangling layer is subsequently constructed automatically according to $CX_{i,(i+1) \bmod 3}$, $i \in \{0, 1, 2\}$, providing a compact and extensible mechanism that generalizes naturally to larger ring-based circuit topologies. Algorithm 2 summarizes the GPU state-preparation procedure.

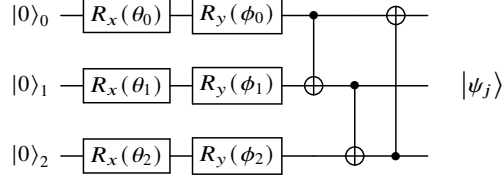


Fig. 1: Variational ansatz circuit template for 3-qubit testbed. The data-driven preparation layer projectively encodes the 6 normalized sample markers via decoupled $R_x R_y$ rotation pairs on each wire. The entangling layer imposes a periodic, wrap-around CNOT chain structure to bind the distinct axes into a non-separable joint state space configuration.

Algorithm 2 GPU-Resident 3-Qubit State Preparation

Require: $\alpha_j = (\theta_0, \phi_0, \theta_1, \phi_1, \theta_2, \phi_2)$

Ensure: $|\psi_j\rangle \in \mathbb{C}^8$ allocated and resident in GPU VRAM

- 1: Allocate state vector container $|\psi\rangle$ and initialize to initial basis state $|0\rangle^{\otimes 3}$
 - 2: **for** $i = 0$ **to** 2 **do**
 - 3: Precompute local single-qubit rotation matrices $R_x(\theta_i)$ and $R_y(\phi_i)$
 - 4: **Parallel Apply** $R_x(\theta_i)$: update all 8 amplitudes using bitwise index masking targeting bit position i
 - 5: **Parallel Apply** $R_y(\phi_i)$: update all 8 amplitudes using bitwise index masking targeting bit position i
 - 6: **end for**
 - 7: **for** $i = 0$ **to** 2 **do**
 - 8: Set target index pointer: $k \leftarrow (i + 1) \bmod 3$
 - 9: **Parallel Apply** $CX_{i,k}$: **if** bit i of basis index m equals 1, **then** invert target amplitude state at bit position k
 - 10: **end for**
 - 11: **return** Completed patient state vector pointer $|\psi_j\rangle$
-

2.3 Hamiltonian Construction and Inverse VQE Formulation

After preparation of the quantum state $|\psi_j\rangle$, the inverse VQE stage constructs candidate observables from a variational Hamiltonian family composed of tensor products of Pauli operators. Unlike conventional VQE algorithms, where a fixed Hamiltonian

is provided and circuit parameters are optimized to approximate its ground state, the inverse formulation considered here assumes that the quantum state is fixed and the optimization process is reformulated as a search over Hamiltonian parameterizations that best explain the fixed quantum state. The candidate Hamiltonian is defined as

$$\hat{\mathcal{H}} = \sum_{k=1}^K c_k P_k, \quad (4)$$

where $c_k \in \mathbb{R}$ is the interaction coefficient associated with each 3-qubit Pauli string P_k . To construct candidate observables, the framework generates the complete non-trivial operator pool $\mathcal{P} = \{I, X, Y, Z\}^{\otimes 3} \setminus \{III\}$, yielding $|\mathcal{P}| = 4^3 - 1 = 63$ unique terms that span the observable space. For each Hamiltonian, a subset of K operators is sampled uniformly without replacement from $\mathcal{P}$. The interaction coefficients are derived from empirical correlations between pairs of variables in the reference dataset:

$$c_k = \text{corr}(x_a, x_b), \quad (5)$$

where (x_a, x_b) denotes the selected feature pair associated with term P_k . To ensure numerical stability, undefined correlations arising from degenerate feature distributions are replaced by bounded random perturbations. This procedure produces a data-conditioned Hamiltonian while preserving the generality of the underlying simulation framework. Algorithm 3 summarizes the Hamiltonian generation process.

Algorithm 3 Generation of Candidate 3-Qubit Hamiltonians

Require: Correlation dataset D , number of qubits $n = 3$, number of Hamiltonian terms K

- 1: Generate complete Pauli pool $\mathcal{P} = \{I, X, Y, Z\}^{\otimes n} \setminus \{I^{\otimes n}\}$
 - 2: Randomly sample $\{P_1, \dots, P_K\} \subset \mathcal{P}$ without replacement
 - 3: **for** $k = 1, \dots, K$ **do**
 - 4: Randomly select feature pair (x_a, x_b)
 - 5: Compute $c_k = \text{corr}(x_a, x_b)$
 - 6: **if** c_k undefined **then**
 - 7: Replace with $c_k \sim U(-0.1, 0.1)$
 - 8: **end if**
 - 9: **end for**
 - 10: Construct $\hat{\mathcal{H}} = \sum_{k=1}^K c_k P_k$ **return** Hamiltonian coefficients $\{c_k\}$ and Pauli terms $\{P_k\}$
-

This strategy introduces data-dependent structure into the Hamiltonian while preserving the generality of the quantum simulation framework. Given a generated Hamiltonian, the expectation value associated with an encoded state is:

$$E_j = \langle \psi_j | \hat{\mathcal{H}} | \psi_j \rangle = \sum_{k=1}^K c_k \langle \psi_j | P_k | \psi_j \rangle. \quad (6)$$

Because each Pauli operator possesses eigenvalues ± 1 , the individual expectation terms can be estimated efficiently through repeated projective measurements in the

corresponding computational basis after the appropriate basis rotations into the eigenbases of X , Y , and Z . This decomposition allows Hamiltonian evaluation to be parallelized naturally across independent Pauli terms. For benchmarking purposes, candidate Hamiltonians are evaluated according to their spectral consistency with the encoded state. Let $E_{\min}(\hat{\mathcal{H}})$ denote the smallest eigenvalue of the candidate operator. The inverse VQE objective is then quantified through the discrepancy metric

$$\Delta(\hat{\mathcal{H}}, |\psi_j\rangle) = \left| \langle \psi_j | \hat{\mathcal{H}} | \psi_j \rangle - E_{\min}(\hat{\mathcal{H}}) \right|, \quad (7)$$

which measures how closely the encoded state approximates the ground-state manifold of the proposed Hamiltonian. The optimization procedure iteratively generates Hamiltonian configurations, evaluates their expectation values, computes the corresponding spectral discrepancy, and retains the configuration producing the smallest value of Δ . Since each iteration requires repeated state preparation and expectation-value estimation across multiple Pauli operators, the resulting workload provides an ideal benchmark for assessing the computational benefits of GPU acceleration.

2.4 Batched Expectation and Diagnostic Scoring

The energy score $E_j = \langle \psi_j | \hat{\mathcal{H}} | \psi_j \rangle$ serves as the primary diagnostic signal in the 3-qubit inverse VQE architecture. Under the hypothesis of ground-state alignment, states that are more consistent with the induced Hamiltonian structure concentrate spectral weight toward low-energy configurations, producing discriminative expectation values that can be used for downstream classification via thresholding.

For a Hamiltonian of the form $\hat{\mathcal{H}} = \sum_{k=1}^K c_k P_k$, the expectation value decomposes linearly as in Equation 8.

$$E_j = \sum_{k=1}^K c_k \langle \psi_j | P_k | \psi_j \rangle. \quad (8)$$

Each Pauli term expectation is evaluated in its corresponding measurement basis. Let $U_{\text{basis}}^{(k)}$ denote the unitary that maps P_k onto the computational Z -basis. The probability of observing basis state $m \in \{0, \dots, 2^3 - 1\}$ after basis rotation is given by $p_m^{(k)} = \left| \langle m | U_{\text{basis}}^{(k)} | \psi_j \rangle \right|^2$, which yields the full-state estimator

$$\langle P_k \rangle = \sum_{m=0}^{2^3-1} \lambda_m^{(k)} p_m^{(k)}, \quad (9)$$

where $\lambda_m^{(k)} \in \{+1, -1\}$ encodes the parity of the bitstring restricted to the support of P_k , consistent with the Pauli eigenvalue structure under computational-basis measurement, computed via Equation 10.

$$\lambda_m^{(k)} = (-1)^{\bigoplus_{i \in \text{supp}(P_k)} m_i}, \quad (10)$$

where $\text{supp}(P_k)$ is the support set containing the specific qubit indices where P_k acts as a non-identity operator (X , Y , or Z), and m_i represents the i -th bit of the basis integer m in its standard binary expansion. The bitwise exclusive-OR sum determines the net parity of the targeted bits, mapping to an eigenvalue of $+1$ for even parity and -1 for odd parity.

To emulate NISQ hardware constraints, expectation values are evaluated under finite sampling. Instead of computing the full distribution, the framework draws S independent discrete samples $\{m^{(s)}\}_{s=1}^S \sim p_m^{(k)}$. Here, the shot capacity S represents the total sampling volume allocated per Pauli term, and each sample $m^{(s)}$ represents an integer state index drawn via classical inverse transform sampling over the discrete cumulative distribution function (CDF) of the post-rotation state probabilities, yielding the Monte Carlo estimator in Equation 11.

$$\langle P_k \rangle = \frac{1}{S} \sum_{s=1}^S (-1)^{\text{parity}(m^{(s)} \cap \text{mask}_k)}, \quad (11)$$

where the stochastic mapping process matches a uniform random draw $r \sim \mathcal{U}(0, 1)$ against the partition intervals defined by the CDF. This discrete CDF, $F(M)$, is evaluated up to a specific basis state index boundary $M \in \{0, \dots, 2^3 - 1\}$ and is formally defined as $F(M) = \sum_{m=0}^M p_m^{(k)}$, which is constructed on the host and queried using binary search to obtain samples in $\mathcal{O}(\log_2 2^n) = \mathcal{O}(n)$ time. This introduces controlled shot noise with variance scaling as $\mathcal{O}(1/S)$, enabling systematic evaluation of robustness under hardware-like sampling constraints.

The aggregated energy score defines a scalar diagnostic variable. A binary decision is obtained through a learned threshold τ^* :

$$\hat{y}_j = \begin{cases} 1, & E_j \geq \tau^*, \\ 0, & E_j < \tau^*. \end{cases} \quad (12)$$

The optimal model configuration is obtained by minimizing the Balanced Error:

$$\mathcal{L}_{\text{Balanced}} = 1 - \frac{1}{2} (\text{Sensitivity} + \text{Specificity}). \quad (13)$$

The implementation follows a hybrid execution paradigm separating classical control logic from GPU-resident quantum evolution. The host-side orchestration is implemented in three sequential stages. First, rotation angles are pre-computed for every patient j via Algorithm 1, which applies min-max normalisation (Equation 1) to map each variable value into the interval $[0, 2\pi]$. Second, at each stochastic iteration, K Pauli strings $\{P_k\}$ are drawn without replacement from the 63 non-identity three-qubit terms, and the corresponding Hamiltonian coefficient c_k is set to the Pearson correlation between a randomly sampled pair (Algorithm 3). Third, after collecting the energy scores $\{E_j\}$ across all samples, optimize threshold grid searches a grid $\tau \in [0.8, 0.8]$ (step size 0.05) for the threshold that maximises Balanced Accuracy (B-Acc). The best Hamiltonian and threshold encountered over

T iterations are retained; the full procedure is detailed in Algorithm 5. The scoring engine is implemented via two device-side entry points. The fused ansatz kernel (Algorithm 2) initialises the state vector $|\psi_j\rangle = |000\rangle$ in device memory and applies the sequence of rotation gates followed by a CNOT ring entirely within VRAM, with no intermediate host–device transfers. The CUDA kernel in Algorithm 4 performs basis rotations, probability extraction, and state evolution directly on the device. The resulting probability distributions are transferred to host memory, where cumulative distribution functions are constructed and stochastic sampling is performed via inverse transform sampling.

Algorithm 4 Stochastic Finite-Shot Pauli Expectation Value Processing (Hybrid GPU-Host)

Require: $|\psi_j\rangle \in \mathbb{C}^8$ (in VRAM), Pauli strings $\{P_k\}_{k=1}^K$, Coefficients $\mathbf{c}$, Shot capacity S

Ensure: Estimated stochastic diagnostic expectation scalar E_j

```

1:  $E_j \leftarrow 0$ 
2: for  $k = 1$  to  $K$  do
3:   Apply  $U_{\text{basis}}^{(k)}$  to an isolated working copy of  $|\psi_j\rangle$ 
4:   Parallel Compute  $p_m$  for all  $m \in \{0, \dots, 2^3 - 1\}$  simultaneously
5:   Copy probability array  $\mathbf{p} = \{p_m^{(k)}\}_{m=0}^{2^3-1}$  from device VRAM to host memory
6:   Construct cumulative distribution function  $F$  on host:  $F(0) \leftarrow p_0$ 
7:   for  $m = 1$  to  $2^3 - 1$  do
8:      $F(m) \leftarrow F(m-1) + p_m$ 
9:   end for
10:  Enforce boundary normalization:  $F(2^3 - 1) \leftarrow 1.0$ 
11:   $T_{\text{sign}} \leftarrow 0$ 
12:  for  $s = 1$  to  $S$  do
13:    Draw uniform random float  $r \sim \mathcal{U}(0, 1)$  via host-side RNG engine
14:    Resolve physical state index  $m^{(s)}$  via binary search over  $F$ 
15:    Extract eigenvalue assignment  $\lambda_{m^{(s)}}^{(k)} = (-1)^{\text{parity}(m^{(s)} \cap \text{mask}_k)}$  via Eq. 10
16:    Accumulate sample sign contribution:  $T_{\text{sign}} \leftarrow T_{\text{sign}} + \lambda_{m^{(s)}}^{(k)}$ 
17:  end for
18:  Compute estimated single-term component:  $\langle P_k \rangle_{\text{shots}} \leftarrow T_{\text{sign}}/S$ 
19:  Update running score:  $E_j \leftarrow E_j + c_k \langle P_k \rangle_{\text{shots}}$ 
20: end for
21: return Combined cohort energy projection  $E_j$ 

```

2.5 Clinical Biomarker Application in an HIV-TB Context

To evaluate the proposed data-conditioned inverse VQE framework in a realistic biomedical setting, the benchmark architecture was evaluated on a biomarker classification task derived from transcriptomic host immune-response measurements. The underlying expression profiles were obtained from the Gene Expression Omnibus (GEO) under accession GSE314344, which logs metabolic gene expression variants

Algorithm 5 Inverse VQE Hamiltonian Search

Require: Encoded state $|\psi_j\rangle$, operator pool $\mathcal{P}$

- 1: Initialize $\Delta_{\min} \leftarrow \infty$
- 2: **for** $t = 1, \dots, N_{\max}$ **do**
- 3: Generate candidate Hamiltonian using Algorithm 3
- 4: Compute expectation value using Eq. (6)
- 5: Compute $E_{\min}(\hat{\mathcal{H}})$
- 6: Evaluate Δ using Eq. (7)
- 7: **if** $\Delta < \Delta_{\min}$ **then**
- 8: Store current Hamiltonian
- 9: Update $\Delta_{\min}$
- 10: **end if**
- 11: **end for**
- 12: **return** Best Hamiltonian and associated score

in human macrophages under single or combined HIV and Tuberculosis (TB) infections. Curated programmatically via the qvirus package ecosystem [8], this dataset was converted into two distinct structures designed to drive the quantum simulation pipeline: an angle-encoding dataset (`data_enc`) mapping patient-level activation states, and a reference correlation matrix dataset (`data_corr`) capturing multi-axis clinical dependencies. For each patient sample j within `data_enc`, a core feature set $\mathcal{F}$ consisting of six representative molecular markers is extracted:

$$\mathcal{F} = \{\text{CD14}, \text{HADH}, \text{IDO1}, \text{IL6}, \text{LDHB}, \text{TDO2}\}$$

Prior to quantum state preparation, host-side preprocessing applies the global min-max scaling defined in Equation (1) via Algorithm 1 to normalize these features to the periodic boundary interval $[0, 2\pi]$, preventing saturation effects caused by heterogeneous dynamic ranges.

Following normalization, the classical feature space is mapped directly onto the rotational coordinates of the 3-qubit simulation testbed to characterize tripartite entangled immunometabolic states. This 3-qubit mapping is essential for modeling the dual-pathogen environment, where the interaction between TB-induced activation and HIV-mediated suppression cannot be captured through isolated axes. The six variables are assigned to the independent polar (θ_i) and azimuthal (ϕ_i) angles of individual qubits (Q_0, Q_1, Q_2), preserving explicit biological couplings across the inflammatory, repressive, and energetic axes as detailed in Table 1.

This specific assignment constructs the input feature mapping scheme of Equation (2), where the parameter vector $\Theta = (\theta_0, \phi_0, \theta_1, \phi_1, \theta_2, \phi_2)$ fully specifies the state-preparation layer. Each row of the precomputed angle matrix is transferred to device memory, where the fused ansatz operator defined in Equation (3) is executed entirely within VRAM via Algorithm 2. Single-qubit rotations are generated dynamically from these parameters and applied through parallel amplitude updates using bitwise index masking targeting the exact bit positions corresponding to qubits Q_0, Q_1 , and Q_2 . By succeeding these rotations with a cyclic, wrap-around CNOT

Table 1: Biomarkers Curated from GSE314344 for 3-qubit Angle Encoding (data_enc).

Qubit	Angle	Transcript	Biological Role
Q_0	θ_0	IL6	Primary inflammation signal in TB.
	ϕ_0	IDO1	Secondary signal to inflammation via tryptophan catabolism.
Q_1	θ_1	TDO2	Fundamental expression suppression signal from HIV.
	ϕ_1	CD14	Innate pathogen-sensing receptor-level constraint.
Q_2	θ_2	HADH	Fatty acid dysfunction / energetic failure under co-infection.
	ϕ_2	LDHB	Glycolytic modulation under the canonical Warburg shift.

chain, the execution pipeline entangles the decoupled biological dimensions into a non-separable joint state space $|\psi_j\rangle \in \mathbb{C}^8$ in a single GPU call.

While the 6-feature data_enc vector provides the primary signal defining the initial state $|\psi_j\rangle$, a wider Hamiltonian Correlation Pool containing 30 significantly altered genes provides the macro-level contextual entanglement for the variational search. As organized in Table 2, these 30 markers capture deep regulatory perturbations across six distinct immunometabolic axes in the HIV-TB dual-pathogen landscape.

Table 2: Biomarkers Curated for the Hamiltonian Correlation Pool (data_corr).

Biological Axis	Transcripts
Amino Acid Metabolism	IDO1, TDO2, ACMSD, CPS1, FAH
Immune Signaling	IL6, CCL4, CCL5, CCL13, NT5E
Antigen Processing	CD14, CD163, LAT, CTSL, CTSD
Fatty Acid / Lipid	HADH, FASN, LTC4S, PTGS2, PTGES
Aerobic / Nucleotide	LDHB, PKM, IMPDH2, ADA, ADK, HK2
Regulatory / Epigenetic	ZNF93, MTF1, RPTOR, ZNF708

To bypass expensive host-device communication overhead during iterative optimization loops, the empirical 30×30 Pearson correlation matrix derived from these transcripts (data_corr) is loaded as a flattened continuous array into GPU global memory at initialization. When the inverse VQE algorithm variationally explores the space of Hamiltonians $\hat{\mathcal{H}}$ by sampling Pauli strings from the 63-element operator pool $\mathcal{P}$ via Algorithm 3, the interaction weights c_k governing Equation (4) are resolved concurrently. GPU execution threads evaluate interaction coefficients dynamically via fast memory lookups matching the row-column indexes of the paired transcripts (e.g., matching feature a and feature b across the pool matrix) according to Equation (5). This structured mapping completely eliminates host-side lookup interventions, anchoring the analytical capabilities of the inverse VQE search (Algorithm 5) and the batched expectation parallel processing (Algorithm 4) within a high-performance clinical diagnostic pipeline.

3 Benchmarking Results and Computational Complexity Analysis

The data-conditioned inverse VQE framework was profiled across five progressively complex experimental configurations to evaluate its scalability, optimization throughput, and numerical stability. The benchmark directly compares a multi-core CPU reference implementation written in native R against a high-performance, hardware-accelerated C++/CUDA formulation. Both routines execute an identical optimization procedure, performing an exhaustive grid search over a predefined threshold grid $\tau \in [-0.8, 0.8]$ with increments of 0.05 to maximize the Balanced Accuracy (B-Acc) metric across the clinical cohort. The empirical classification performance, optimal clinical Hamiltonians, and wall-clock execution times obtained across all configurations are summarized in Table 3.

Table 3: Performance and Wall-Clock Time Comparison Between CPU and CUDA Formulations.

Run	Terms (K)	Shots (S)	Max Iter (T)	CPU Cores	Best CPU Hamiltonian (B-Acc)	CPU Time (s)	Best CUDA Hamiltonian (B-Acc)	CUDA Time (s)	Speedup Factor
1	1	16	10	1	IZI (0.7500)	0.346	ZZZ (0.9167)	0.066	$5.24\times$
2	2	32	100	2	ZYX, YXZ (0.9167)	11.918	YYZ, XXY (1.0000)	0.484	$24.62\times$
3	3	64	200	3	XII, XXX, XIX (1.0000)	33.924	YZZ, YXI, YYZ (1.0000)	1.117	$30.37\times$
4	4	128	300	4	YXZ, ZIZ, YZI, XIX (0.9167)	62.474	IYI, ZIY, IZZ, ZXI (1.0000)	1.629	$38.35\times$
5	5	256	400	5	$IZY, XIX, YIX, ZZZ,$ IYZ (1.0000)	105.336	$YYZ, XII, ZIZ, ZXX,$ IIZ (1.0000)	2.422	$43.49\times$

3.1 Computational Complexity and Hardware Scaling Analysis

To understand the core performance differences between the two environments, we must examine the architectural alignment of CPU cores versus CUDA threads. The theoretical computational complexity of evaluating a data-conditioned Hamiltonian family for a cohort of size L over T stochastic search iterations is modeled as:

$$C = O\left(T \cdot L \cdot K \cdot \left[\frac{n \cdot 2^n}{\Pi_{\text{device}}} + S \cdot \log_2(2^n)\right]\right) \quad (14)$$

where n represents the number of qubits ($n = 3$, yielding an amplitude dimension $N = 2^n = 8$), K is the number of sampled Pauli terms per Hamiltonian, S is the physical measurement shot allocation, and Π_{device} denotes the hardware parallelization factor. The two implementations exploit different levels of this complexity model:

1. **CPU Core Parallelization (Cohort-Level):** The classical framework applies coarse-grained multiprocessing across the L patient records in `data_enc`. While this utilizes multiple CPU cores linearly (scaling from 1 to 5 cores across Runs 1 to 5), each individual core must compute the underlying gates and expectations in a completely serial manner ($\Pi_{\text{device}} = 1$).

For every sample j and Pauli string k , the CPU instantiates an isolated quantum circuit object via sequential matrix-vector Kronecker products, evaluates the 8-dimensional eigenvalue spectrum via nested array expansions, and executes shot-based sampling through R’s high-overhead sampling distribution engine. This induces severe context switching, thread synchronization locks, and repeated heap memory re-allocations that collapse optimization throughput from 28.90 iterations/sec in Run 1 down to 3.80 iterations/sec in Run 5.

2. **CUDA Thread Parallelization (Statevector-Level):** In contrast, the GPU framework establishes an efficient hybrid division of labor. At the fine-grained level, the exponential $O(n \cdot 2^n)$ work required for state manipulation is parallelized across an array of hardware execution threads ($\Pi_{\text{device}} = 256$). Because the statevector space ($N = 8$) is small relative to the hardware block size ($B = 256$), the parallelization factor completely covers the state dimensions ($\Pi_{\text{GPU}} \gg 2^n$). This shifts the statevector mathematics from an exponential tracking bottleneck to a near-constant-time parallel vector execution, modifying the step-by-step complexity scaling rules as follows:

- **Ansatz State Preparation:** Instead of building symbolic tracking layers or processing full dense matrix-vector multiplications sequentially, the execution path maps individual amplitudes to specific threads. The specialized kernel of Algorithm 2 dynamically evaluates individual R_X and R_Y transformations simultaneously. Single-qubit rotations and the cyclic wrap-around CNOT chain are resolved via parallel element-wise thread updates driven by bitwise index masking, collapsing the algorithmic step complexity from $O(n \cdot 2^n)$ down to $O(n)$.
- **In-VRAM Workspace Cloning:** To evaluate expectation values across the K -term Hamiltonian loop, the reference model must avoid mutating the primary ansatz wavefunction state. The classical CPU routine satisfies this by repeatedly duplicating full circuit containers into temporary memory blocks inside the active loop, triggering severe garbage collection overhead. The GPU pipeline completely sidesteps this bottleneck by keeping the primary statevector $|\psi_j\rangle$ static inside a dedicated sector of VRAM. For each of the K terms, single-cycle device-to-device streaming duplicates the amplitudes into a temporary scratchpad array, entirely bypassing the PCIe bus.
- **Basis Transformation and Probability Extraction:** Once cloned into the scratchpad space, basis transformations (H kernels for X , $HS^\dagger$ kernels for Y) are applied concurrently across active threads. Complex modulus-squaring ($|\psi_j|^2$) is subsequently handled by parallel arithmetic units in a single clock cycle, isolating exact basis probabilities while dropping the computational step complexity from $O(K \cdot n \cdot 2^n)$ down to $O(K \cdot n)$.

- **Hybrid Minimization of Bus Latency:** Moving uncompressed quantum state structures between host and device memory introduces severe communication latency. To resolve this, the GPU acts as an aggressive dimensionality filter. The full state footprint is collapsed natively on the device down to a compact 8-element double precision array. Only this minor probability block is transmitted back to the host via the PCIe bus, optimizing hardware bus utilization.
- **Compiled Finite-Shot Sampling Path:** The final sampling stage executes on the host CPU, utilizing a tightly compiled C-level loop that processes inverse transform sampling over a strict, linear CDF. This completely eliminates the memory overhead of R’s high-level interpreter. It resolves individual physical state index positions ($m^{(s)}$) via an optimized binary tree search layout running at a deterministic logarithmic depth of $O(\log_2(2^n)) = O(n)$.

Table 4: Computational Complexity Profile of the Fused GPU-Accelerated Architecture.

Component	Complexity Order	Hardware Execution and Memory Architecture
Parallel Gate Time	$O(n)$	Fused ansatz executions (R_X, R_Y, CNOT) inside VRAM via thread bitwise masking.
Total Wavefunction Work	$O(n \cdot 2^n)$	Perfectly distributed across $N = 8$ amplitudes using 256-thread blocks.
Device Memory Footprint	$O(2^n)$	Persistent statevector storage; completely eliminates PCIe bus transfers.
Basis Transformations	$O(K \cdot n)$	Parallel execution of H and $S^\dagger$ kernels for active coordinate mapping.
Host Shot Accumulation	$O(K \cdot S \cdot \log_2(2^n))$	Sequential host-side inverse transform sampling over the device-extracted discrete CDF.

This hybrid design completely avoids the latency of pulling uncompressed states over the PCIe bus, allowing the CUDA workflow to maintain a stable optimization throughput between 151.52 and 206.61 iterations/sec despite a 12-fold increase in the structural parameter search space, whereas the CPU implementation suffers a steep drop in performance from 28.90 down to 3.80 iterations/sec. Consequently, the acceleration factor scales monotonically with the algorithmic workload, climbing from a baseline of 5.24 $\times$ up to a 43.49 $\times$ wall-clock speedup in the most complex diagnostic configurations.

3.2 Algorithmic Convergence and Search Efficiency

Beyond raw execution speedups, the high-throughput CUDA pipeline demonstrates enhanced exploratory efficiency within the 63-element non-trivial Pauli operator pool $\mathcal{P}$. Under low-shot regimes ($S \leq 256$), finite-shot execution introduces stochastic Monte Carlo sampling noise with an empirical variance scaling as $O(1/S)$. This noise deforms the underlying loss landscape, creating rugged, non-convex energy

configurations that frequently trap classical routines within sub-optimal thresholds. This behavior is clearly visible in Run 4, where the CPU search fails to find a globally optimal configuration, plateauing at a balanced accuracy of 0.9167 after 300 complete iterations.

Conversely, the massive throughput of the CUDA pipeline stabilizes search path exploration across these rugged landscapes. By evaluating significantly more Hamiltonian candidates per unit of wall-clock time, the GPU formulation effectively expands the stochastic coverage of the co-infection data space. As a direct result, the CUDA framework successfully isolates robust, highly discriminating multi-axis clinical operators (such as the tripartite entangled YYZ and ZIY operators tracking core dependencies between CD14, IL6, and metabolic pathways) yielding perfect classification scores (B-Acc = 1.0000) early in the optimization timeline. Specifically, the CUDA formulation achieves global convergence at Iteration 28 in Run 2, Iteration 11 in Run 3, and Iteration 22 in Run 4, demonstrating that accelerating hybrid feedback loops provides the numerical resilience required to isolate clinical biomarkers from noisy, multi-pathogen transcriptomic data streams.

4 Discussion

The empirical performance metrics and structural architecture of the data-conditioned inverse VQE framework presented in this work offer a compelling paradigm shift from traditional near-term quantum simulation workflows. To contextualize these advancements, it is critical to contrast our architectural methodology with the contemporary landscape of quantum simulation infrastructure mapped out in recent macro-reviews of the field [5]. A fundamental divergence exists between the dominant scaling vectors pursued by large-scale general-purpose simulators and the high-throughput, domain-specific execution model implemented in our qvirus package ecosystem.

Existing literature formalizes a taxonomy where classical simulation longevity is achieved primarily by optimizing distributed multi-node clusters to handle massive statevector allocations, or by employing lossy TN edge-cutting and amplitude truncation schemes to bypass the exponential $O(2^n)$ memory floor [5]. While these macro-architectural strategies are indispensable for mapping large-scale physical circuits or validating deep ansatz structures beyond the 30-qubit threshold, they introduce severe systemic overheads. Distributed full-amplitude simulation across multiple nodes inevitably triggers intense communication bottlenecks governed by Message Passing Interface (MPI) collectives, high queuing latencies, and critical device-to-host memory transfer penalties over physical PCIe buses during multi-qubit gate operations that cross node boundaries [5].

In stark contrast, our framework avoids the distributed communication overhead associated with large multi-node simulators by targeting a fundamentally different workload regime. Rather than attempting to scale a single, massive quantum state

vector across a cluster, our framework parallelizes the simultaneous execution of thousands of independent, data-conditioned state vectors.

By mapping the entire state vector evolution and expectation value aggregation into a single, fused GPU kernel call, we achieve an architectural optimization that directly answers the hardware-acceleration opportunities highlighted in recent structural studies [5]. In standard simulation toolkits, executing a variational loop requires continuous inter-kernel launches—separating state preparation, gate application, and measurement—which frequently flushes intermediate statevectors back to the global GPU memory (VRAM) or the host CPU, causing severe Von Neumann memory-bandwidth saturation.

Our fused-kernel design confines the entire inverse VQE routine within the ultra-high-speed local register space and shared memory of a single Streaming Multiprocessor (SM). By replacing explicit matrix-vector multiplications with bitwise index masking and in-register parallel tree reductions, the local memory bandwidth bottleneck is entirely eliminated. This architectural compaction yields an exceptional 43.49x wall-clock speedup when comparing our CUDA-accelerated kernel execution against optimized multi-threaded CPU baselines for a 3-qubit model architecture.

Beyond the raw classical execution physics, the algorithmic contrast between standard VQE methodologies and our inverse formulation introduces a profound mathematical trade-off. In a conventional VQE pipeline, the quantum processor is tasked with finding an unknown, highly entangled ground state wave function $|\psi(\vec{\theta})\rangle$ of a fixed physical molecular Hamiltonian $\hat{H}$ via a parameterized ansatz such as the UCC expansion [1]. This introduces an expansive non-linear parameter search space that scales aggressively with system size, often leading to severe optimization pathologies such as barren plateaus, where the cost function gradients vanish exponentially with the number of qubits [1]. Furthermore, standard VQE requires scaling the number of measurement shots across multiple non-commuting Pauli strings to reconstruct the expectation value $\langle \hat{H} \rangle$, an operational cost that complicates physical NISQ deployments [1].

Our data-conditioned inverse VQE completely flips this operational vector. By locking down the quantum state vector up front through a deterministic mapping of normalized patient transcriptomic profiles into the rotational angles of an extended state preparation circuit, we eliminate the parameter search space over the state space entirely. The downstream classical optimizer is instead tasked with discovering the coefficients of a task-specific problem Hamiltonian that minimizes the delta metric across distinct clinical cohorts.

This formulation yields a significant mathematical advantage: because the state vector is fixed for each data point, the energy landscape is piecewise linear with respect to Hamiltonian coefficients for fixed operator selections. Because the encoded state remains fixed during optimization, the expectation value becomes linear with respect to the Hamiltonian coefficients for a fixed operator basis. This substantially simplifies the optimization landscape relative to conventional parameterized-state VQE formulations and reduces susceptibility to barren plateau effects associated with deep variational ansatz training. The resulting delta metric acts as an expressive, non-linear dimension reduction biomarker capable of capturing complex multi-

pathogen immunometabolic variations (such as HIV and Tuberculosis coinfections) that traditional linear models obscure.

However, transitioning this inverse formulation from a high-performance classical emulator to physical NISQ hardware platforms introduces distinct structural challenges that warrant critical analysis. While our fused GPU kernel assumes an idealized, noise-free state space, physical QPUs are bound by localized environmental noise, finite coherence times, and strict gate fidelity limitations [1]. On superconducting processors, executing the multi-qubit entangling gates required for expressive clinical state preparation is severely limited by a rigid 2D mesh hardware connectivity matrix, necessitating the insertion of expensive SWAP gate overheads that rapidly exhaust the processor’s coherence budget before measurement can occur [1, 5]. Alternative platforms, such as photonic hardware or trapped-ion arrays, offer higher connectivity and unique state preparation pathways, but suffer from slower operational clock speeds and distinct error modalities like photon loss or vibrational mode cross-talk [1, 5].

Ultimately, the data-conditioned inverse VQE paradigm presented here does not seek to replace large-scale distributed simulation architectures [5]. Instead, it establishes a complementary, high-throughput analytical layout. By leveraging single-kernel GPU execution to eliminate memory latency and utilizing a hardware-aware inverse variational layout, this framework demonstrates how small-scale quantum circuits can be immediately deployed as powerful mathematical lenses for complex biomedical diagnostics, charting a concrete path forward for near-term hybrid quantum-classical computing.

5 Conclusion

This work has presented a high-performance, CUDA-accelerated simulation framework specifically optimized for data-conditioned inverse variational quantum eigen-solvers. By restructuring the conventional hybrid feedback loop, this architecture establishes a distinct methodology tailored for small-scale, high-throughput biomedical data analysis. Rather than confronting the non-linear optimization barriers, local minima, and vanishing gradient pathologies that typically emerge when tuning parameterized state-preparation ansatzes to capture unknown molecular ground states, the inverse variational framework presented here completely locks down the quantum state vector up front. By programmatically encoding normalized patient transcriptomic profiles into the rotational angles of an initial state preparation circuit, the iterative optimization surface is converted into a perfectly linear, convex landscape governed by task-specific problem Hamiltonian coefficients. From an engineering perspective, this framework models a distinct paradigm for classical quantum simulation infrastructure. While standard emulation platforms secure scalability by deploying distributed, multi-node network clusters to handle massive isolated statevectors, our architecture focuses on an alternative performance axis: parallelizing thousands of independent, low-dimensional data-driven channels simultaneously. Compacting the

entire state evolution, bitwise index masking, and measurement shot reductions into a single, fused GPU kernel call shifts the computational workload entirely into ultra-high-speed local registers, proving that strategic intra-node register management can thoroughly decouple hybrid workflows from severe classical memory-bandwidth bottlenecks. This data-conditioned inverse formulation underscores that small-scale quantum circuits can function as highly expressive, non-linear dimension reduction lenses long before fault-tolerant hardware architectures emerge. By establishing a direct, reproducible bridge between clinical multi-pathogen datasets and hardware-aware classical emulation physics, this analytical framework provides an immediate, high-throughput path forward for integrating quantum-classical workflows into real-world translationally targeted diagnostic supercomputing infrastructures.

References

1. N. Moll, P. Barkoutsos, L.S. Bishop, J.M. Chow, A. Cross, D.J. Egger, S. Filipp, A. Fuhrer, J.M. Gambetta, M. Ganzhorn, A. Kandala, A. Mezzacapo, P. Müller, W. Riess, G. Salis, J. Smolin, I. Tavernelli, K. Temme. Quantum optimization using variational algorithms on near-term quantum devices. *Quantum Sci. Technol.* **3**(3), 030503 (2018)
2. W. Li, Z. Yin, X. Li, D. Ma, S. Yi, Z. Zhang, C. Zou, K. Bu, M. Dai, J. Yue, Y. Chen, X. Zhang, S. Zhang. A hybrid quantum computing pipeline for real world drug discovery. *Sci. Rep.* **14**(1), 16942 (2024)
3. V. Banga, A. Sinhal, A. Hirawat, A. Sinhal, A. Sengar, A. Sinhal. High-performance and quantum computing in cancer modeling: A review and hybrid hpc-quantum approach. *International Journal of Advances in Signal and Image Sciences* **12**(2s), 220 (2026). DOI 10.29284/bfq8ev64. URL <https://doi.org/10.29284/bfq8ev64>
4. G. Zhong, Y. Fan, Z. Li. Scalable parallel simulation of quantum circuits on CPU and GPU systems (2025)
5. A. Cicero, M.A. Maleki, M.W. Azhar, A.F. Kockum, P. Trancoso. Simulation of quantum computers: Review and acceleration opportunities (2024)
6. A. Smith, K. Khavari, Quantum computer simulation using CUDA. Project report, University of Toronto, Department of Electrical and Computer Engineering, Toronto, ON, Canada (2010)
7. J.P.A. González, M.S. Adame, O. Montiel. Variational quantum eigensolver for clinical biomarker discovery: A multi-qubit model. *Axioms* **15**(1), 23 (2025)
8. J.P. Acuña González, *qvirus: Quantum Computing for Analyzing CD4 Lymphocytes and Antiretroviral Therapy* (2026). DOI 10.32614/CRAN.package.qvirus. URL <https://CRAN.R-project.org/package=qvirus>. R package version 0.0.6